SCS1303

Introduction to Software Engineering

Section 6 · Implementation

Prof. K. P. Hewagamage

B.Sc. Computer Science · Year 1, Semester 1

University of Colombo School of Computing

Section 6 — Intended Learning Outcomes

By the end of this section, students should be able to:

LO1	Explain the role of implementation in the software lifecycle and its relationship to design
LO2	Identify major programming paradigms and apply appropriate criteria to select a language for a project
LO3	Describe the professional development environment — IDEs, package managers, linters, and formatters
LO4	Apply good coding practices — reliability, readability, reuse, and the SOLID principles
LO5	Conduct and participate in a code review — walkthrough, inspection, and pull request workflows
LO6	Use Git for version control — commit, branch, merge, push, pull, and collaborate via GitHub
LO7	Explain open source licensing and identify the most common licence models and their implications
LO8	Describe how AI tools change the implementation paradigm and what professional responsibilities engineers retain

Section 6 — Roadmap

6.1

What is Implementation?

6.2

Programming Languages Today

6.3

The Development Environment

6.4

Good Coding Practices

6.5

Code Quality and Review

6.6

Version Control with Git

6.7

Open Source and Licensing

6.8

AI in Implementation

Section 6.1

What is Implementation?

Transforming design into working software

What is Implementation?

Definition

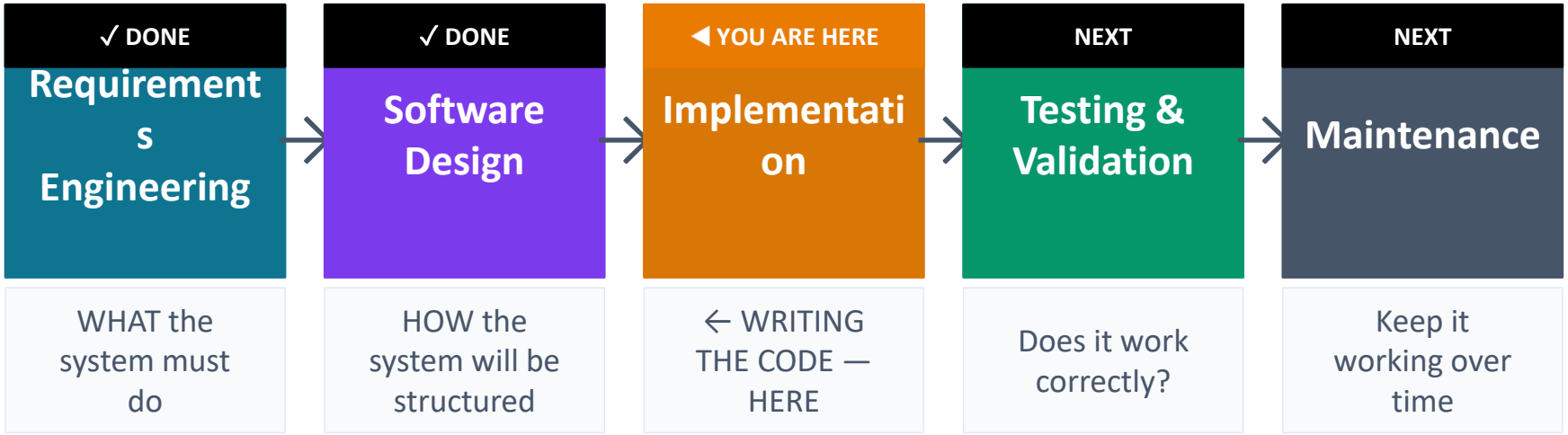
Implementation is the process of translating a software design specification into executable source code. It bridges the gap between what a system should do (design) and a working system that actually does it.

Three key questions implementation must answer:

Which language?	Choose the most appropriate programming language for the problem, the team, and the environment
How to write it?	Apply good coding practices — readable, reliable, maintainable, and reusable
How to manage it?	Use tools — version control, IDEs, and testing frameworks — to keep quality high as the team grows

Implementation in the Lifecycle

Where we are now — and what connects to this stage



Implementation directly affects BOTH testing and maintenance:

Code that is hard to read is hard to test — testers cannot tell what the code is supposed to do. Code that is hard to understand is expensive to maintain — the maintenance team must re-learn it from scratch. Good implementation is an investment in every phase that follows.

A Misconception About Implementation

Coding is not the biggest or most important activity in software development

Common misconception: software engineering = writing code

In reality, coding is a small fraction of the total software project cost and time:

Requirements and design	**25–35%**
Implementation (coding)	**15–20%**
Testing and QA	**25–30%**
Maintenance over lifetime	**60–80% of total lifecycle cost**

Time spent writing code is small — but the quality of that code determines the cost of everything that comes after it.

Section 6.2

Programming Languages Today

Paradigms, the modern landscape, and how to choose

A Brief History — How Languages Evolved

Each generation solved a problem the previous one could not

1GL — **Machine Code · 1940s–50s**
Binary instructions — zeros and ones — the only language a computer understands natively. Impossible for humans to write reliably at scale.

2GL — **Assembly Language · 1950s–60s**
Human-readable codes (ADD, MOV, JMP) that map directly to machine instructions. Faster to write than binary but still tied to specific hardware.

3GL — **High-Level Languages · 1960s–present**
C, Pascal, Java, Python — hardware-independent. Structured programming. One line of high-level code compiles to many machine instructions.

4GL — **Domain-Specific Languages · 1970s–present**
SQL for databases, MATLAB for mathematics, R for statistics. Tailored for one domain — very productive within it, limited outside it.

5GL — **Declarative and AI Languages · 1980s–present**
You describe WHAT you want, not HOW to compute it. Lisp, Prolog, and increasingly — natural language prompts to AI systems.

Programming Paradigms

The fundamental approaches to how we instruct a computer

A paradigm is a style or way of thinking about programming. Most modern languages support multiple paradigms.

Procedural / Structured

Write a sequence of instructions. Use functions to organise and reuse code. Execute step by step, top to bottom.
Examples: C, Pascal, early BASIC
Still widely used in: scripting, systems programming

Object-Oriented (OOP)

Organise code as objects — data and the methods that act on it. Encapsulation, inheritance, and polymorphism.
Examples: Java, C++, C#, Python (also OOP)
Still dominant in: enterprise systems, Android, game development

Functional

Functions are the primary building blocks. Avoid shared state and mutable data. Pure functions, no side effects.
Examples: Haskell, Erlang, Scala; features in Python, JavaScript
Used in: data pipelines, concurrent systems, financial computing

Declarative

Describe WHAT result you want, not HOW to compute it. The runtime figures out the how.
Examples: SQL (query what data), HTML (declare structure), React (declare UI state)
Growing with: AI prompting — you describe the outcome, the model generates the code

The Modern Language Landscape — 2026

What engineers are actually using today — and why

No single language dominates all domains. The choice depends on the problem, the team, and the ecosystem.

Python

General purpose, data science, AI/ML, scripting, rapid prototyping. Beginner-friendly. Enormous library ecosystem (pip). Most popular language in the world in 2026.

JavaScript / TypeScript

Web development — front-end and back-end. Runs in every browser. TypeScript adds strong typing. Used by almost every web company. Node.js for server-side.

Java / Kotlin

Enterprise applications, Android mobile development. Strongly typed, JVM ecosystem. Kotlin is Java's modern successor — cleaner, safer.

C / C++

Systems programming, embedded systems, game engines, operating systems. Closest to hardware. Fast but requires careful memory management.

C#

Microsoft ecosystem, game development (Unity), enterprise .NET. Similar to Java but with more modern language features.

Go (Golang)

Cloud infrastructure, microservices, network tools. Fast to compile, concurrent by design. Used extensively at Google, Kubernetes, Docker.

Rust

Systems programming where safety matters. Prevents memory bugs at compile time. Growing rapidly for security-critical code.

SQL

Every application that stores data uses SQL. Not just a query language — essential literacy for every software engineer.

Choosing the Right Language

Technical and non-technical factors both matter — and non-technical ones usually win

Technical factors

- **Domain fit:** Is the language designed for this type of problem? Python for AI, SQL for databases, JavaScript for web
- **Performance:** Does the application need raw speed? C/C++/Rust for speed-critical systems; Python for data science
- **Safety and security:** Safety-critical systems need strong typing and memory safety guarantees
- **Platform and interfaces:** Does it need to run on mobile, web, embedded device? Does it interface with existing systems?
- **Library ecosystem:** Are the libraries you need available? pip/npm/Maven availability matters enormously

Non-technical factors (often more decisive)

- **Team expertise:** Use what the team already knows — retraining costs time and money
- **Timescales:** A familiar language with good tooling beats a perfect language the team must learn under deadline
- **Maintenance:** The maintenance team must understand the language years from now
- **Organisation standards:** Many organisations standardise on one or two languages across all projects
- **Morale:** Developers who enjoy the language are more productive — but this should not outweigh other factors on critical projects

No language is right for everything.
The best language is usually the one the team knows well, with good libraries for the domain.

Section 6.3

The Development Environment

The tools that professional engineers use every day

The Modern Development Environment

A professional engineer uses many tools together — not just a text editor

What is a development environment?

A development environment is the complete set of tools a developer uses to write, test, manage, and deploy software. Choosing and configuring good tools makes the whole team faster and the code better.

The five layers of a modern development environment:

IDE	Integrated Development Environment — the workspace where code is written, run, and debugged
Package manager	Manages the libraries and dependencies your project needs — pip, npm, Maven
Version control	Git — tracks every change, enables collaboration and safe experimentation
Linter and formatter	Automatically checks code quality and enforces style — catches errors before running
Testing framework	Runs automated tests to verify the code works — JUnit, pytest, Jest

Integrated Development Environments (IDEs)

More than a text editor — an IDE understands your code

What makes an IDE different from a text editor?

An IDE understands the programming language — it can autocomplete, detect errors before you run the code, navigate between files, and connect to a debugger. A text editor is just a word processor for code.

VS Code	Microsoft · Free · Works with any language via extensions · The most popular IDE in the world · Built-in Git, terminal, debugger, and AI (Copilot)
IntelliJ IDEA	JetBrains · Gold standard for Java and Kotlin · Deep code analysis · Used in most enterprise Java development
PyCharm	JetBrains · Best IDE for Python · Data science tools, virtual environments, Jupyter notebook support
Xcode	Apple · Required for iOS and macOS development · Swift and Objective-C
Visual Studio	Microsoft · Powerful for C# and C++ · Enterprise-scale Windows and .NET development

Package Managers — Managing Dependencies

Every modern project relies on libraries written by others

What is a dependency?

A dependency is a library or package that your code relies on — code someone else has written that you include in your project. Package managers download, install, and update these dependencies automatically.

Why package managers matter:

✗ Without a package manager

Manually download every library · Track versions yourself · Tell every new developer to install each one manually · One wrong version breaks everything

✓ With a package manager

One command installs everything · Version numbers are recorded in a file · New developers run one command to get the whole project set up

Common package managers by language:

pip: Python — pip install requests · requirements.txt records all dependencies

npm: JavaScript/Node — npm install axios · package.json is the dependency file

Maven / Gradle: Java — declared in pom.xml or build.gradle

Linters and Code Formatters

Automated tools that enforce quality and consistency

Linter

Analyses your code for errors, bugs, and style problems WITHOUT running it. Catches issues before they become runtime failures.

Examples:

- ESLint — JavaScript
- Pylint / Flake8 — Python
- Checkstyle — Java
- SonarQube — any language, deep analysis

Code Formatter

Automatically rewrites your code to follow a consistent style — spacing, line length, bracket placement. Eliminates style debates in teams.

Examples:

- Prettier — JavaScript, CSS, HTML
- Black — Python (opinionated, zero config)
- gofmt — Go (built into the language)
- clang-format — C, C++

Both tools run automatically on every save or commit — they catch problems immediately, not in a code review hours later.

Section 6.4

Good Coding Practices

Reliability, readability, reuse — and the principles that tie them together

Three Pillars of Good Code

The goal of good implementation

Code that works is the minimum. Good code works, is easy to understand, is unlikely to fail unexpectedly, and can be modified by people other than the original author.

Three properties every engineer aims for:

1. **Reliability**

   Is the software fault-free? Does it continue to work when unexpected things happen?

2. **Readability**

   Can someone else understand, review, and modify this code without asking the author for help?

3. **Reuse**

   Can components be used again in this project or in future projects without rewriting from scratch?

Reliability — Writing Code That Does Not Fail

Two strategies: prevent faults from entering, and handle faults gracefully when they do

Fault Avoidance

Aim to write code that contains no errors in the first place.

- Follow coding standards and naming conventions — reduces ambiguity
- Use strong typing — the compiler catches type errors before the code runs
- Keep functions small and focused — easier to reason about and test
- Write unit tests first (TDD) — forces you to think about behaviour before writing logic
- Peer review and pair programming — another pair of eyes catches mistakes early

Fault Tolerance

Accept that some errors will occur and write code that handles them gracefully.

- Exception handling — catch expected errors and respond appropriately, do not crash
- Defensive programming — validate all inputs before using them, assume inputs may be wrong
- Meaningful error messages — when something fails, say what failed and why clearly
- Graceful degradation — if one part fails, can the rest of the system continue?
- Logging — record errors so they can be diagnosed and fixed after the fact

Exception Handling — An Example

When something goes wrong, handle it — do not crash

Exception handling lets a program respond to unexpected errors without crashing. Modern languages build this in as a feature.

Student Portal example — fee payment with error handling:

Python — fee payment with try / except / finally

```
def process_fee_payment(student_id, amount):
    try:
        student = database.get_student(student_id)   # Might fail if ID invalid
        if amount <= 0:
            raise ValueError("Payment amount must be positive")
        receipt = payment_gateway.charge(student, amount)  # Might fail if gateway down
        database.record_payment(student_id, receipt)
        return {"status": "success", "receipt": receipt}
    except StudentNotFoundError:
        return {"status": "error", "message": "Student ID not found"}
    except ValueError as e:
        return {"status": "error", "message": str(e)}
    except PaymentGatewayError:
        return {"status": "error", "message": "Payment gateway unavailable, please try later"}
    finally:
```

The 'finally' block always runs — use it to release resources like database connections, whether or not an error occurred.

Readability — Code That Others Can Understand

You write code once. It is read dozens of times — by reviewers, testers, and future maintainers

A readable program is not just written for the computer — it is written for the next human who reads it.

Naming — the most important readability decision you make

Python — hard to understand

```
# ✗ Poor naming
def calc(x, lst):
    t = 0
    for i in lst:
        t += i.f * x
    return t
```

Python — immediately clear

```
# ✓ Good naming
def calculate_total_fees(tax_rate, enrollments):
    total = 0
    for enrollment in enrollments:
        total += enrollment.fee * tax_rate
    return total
```

- Use full, meaningful names — never abbreviate unless the abbreviation is universally known (e.g. url, id, max)
- Function names should say what they DO — calculate_total_fees( ), not calc( )
- Variable names should say what they ARE — student_enrolment_count, not n or cnt
- Constants in UPPER_CASE — MAX_ENROLMENTS, not max or m

Comments and Self-Documenting Code

When to comment — and when to let the code speak for itself

The best code is self-documenting — it is so clearly written that comments are rarely needed to explain WHAT it does. Comments explain WHY.

Comment rules:

- Comment WHY, not WHAT — the code shows WHAT; explain the reasoning behind non-obvious decisions
- Write docstrings for every function — document its purpose, parameters, return value, and errors
- Keep comments up to date — an outdated comment is worse than no comment at all
- If code needs a long comment to explain what it does, the code itself needs to be rewritten

Python — comment examples

```
# ✗ Useless comment — describes what the code obviously does (waste of a line)
total = total + fee   # Add fee to total


# ✓ Useful comment — explains WHY a non-obvious decision was made
# Late fee is waived for the first offence per semester policy (regulation 4.2.1)
if is_first_late_payment_this_semester(student_id):
    late_fee = 0


# ✓ Docstring — documents what a function does, its parameters, and its return value
def calculate_gpa(grades: list[float], credits: list[int]) -> float:
    """Calculate GPA using credit-weighted average.
    Args: grades — list of grade points per course; credits — corresponding credit hours
    Returns: float — weighted GPA rounded to 2 decimal places
    Raises: ValueError if grades and credits lists differ in length
    """
```

SOLID Principles —Introduction

Five principles for writing code that is easy to change and extend

What are the SOLID principles?

SOLID is a set of five design principles for writing object-oriented code that is easy to maintain, extend, and understand. Each principle targets a common cause of fragile, hard-to-change code.

S	**Single Responsibility**	A class or function should have ONE reason to change — it should do one thing only
O	**Open / Closed**	Open for extension, closed for modification — add new behaviour without changing existing code
L	**Liskov Substitution**	Subclasses should be substitutable for their parent class without breaking the program
I	**Interface Segregation**	Many specific interfaces are better than one large general-purpose interface
D	**Dependency Inversion**	Depend on abstractions, not on concrete implementations — use interfaces, not specific classes

SOLID principles connect directly to the design principles — cohesion, coupling, and information hiding in action.

Technical Debt — The Hidden Cost of Shortcuts

Every shortcut you take now will cost more later

What is technical debt?

Technical debt is the future cost of additional work created by choosing an easy solution now instead of a better, more complete one. Like financial debt, it accumulates interest — the longer you leave it, the more expensive it becomes to fix.

Common ways technical debt accumulates:

Skipping tests:	'We'll add tests later' — later never comes. Every new feature built on untested code multiplies the debt.
Copy-paste programming:	Duplicated code means every bug fix and change must be made in multiple places — and one is always forgotten.
Ignoring linter warnings:	Each ignored warning is a known problem left in the code. They compound into a codebase nobody wants to touch.
Hard-coding values:	Putting specific values (magic numbers, URLs, paths) directly in code — breaks when environment changes.

Refactoring — improving the internal structure of existing code without changing its behaviour — is the primary way to repay technical debt.

Section 6.5

Code Quality and Review

Catching problems before they reach production

Why Code Review Matters

Code written by one person — checked by another. This is how quality is built.

What is a code review?

A code review is a process in which one or more developers examine code written by another developer — looking for errors, design problems, and improvements before the code is merged into the main codebase. It is the most cost-effective quality activity in software development.

What code review achieves:

Catches bugs early	Defects found in review cost 10× less to fix than defects found after deployment
Spreads knowledge	Reviewers learn the codebase; authors learn from feedback — no single point of knowledge
Enforces standards	Naming conventions, architecture decisions, and quality standards are applied consistently
Improves design	A reviewer often sees a simpler or cleaner solution the author missed while deep in the problem
Builds team culture	Regular review creates a culture of craftsmanship and shared responsibility for quality

Code Review — Two Formal Techniques

Walkthrough and Inspection are structured approaches used in plan-driven development

Code Walkthrough

Informal · 3–7 team members · Author presents the code

- Members receive the code a few days before the meeting and prepare test cases
- During the meeting, members trace through the code manually with their test cases
- Team discusses issues found — the author listens and takes notes
- Goal: discover algorithmic and logical errors, not assign blame
- Output: list of issues for the author to fix

Code Inspection

Formal · Structured roles · Uses a checklist

- Roles: Moderator, Author, Reviewer(s), Recorder
- Code is read line by line against a defect checklist
- Common checklist items: uninitialized variables, array bounds, infinite loops, memory leaks
- All defects are recorded by the recorder — author does not defend the code
- After inspection, author fixes defects and may need re-inspection

Code review is NOT about criticising the person — it is about improving the code. Separate the code from its author.

Pull Requests — The Modern Code Review

How teams review code in practice today — on GitHub, GitLab, and Bitbucket

What is a Pull Request (PR)?

A pull request is a formal request to merge code from one branch into another. Before merging, team members review the changes, leave comments, and the author addresses them. It is the standard code review workflow in virtually all modern development teams.

The Pull Request workflow:

1. Developer writes code on a feature branch — not directly on main
2. Developer pushes the branch to GitHub and opens a Pull Request
3. One or more reviewers are assigned — they see every changed line
4. Reviewers leave inline comments on specific lines or general feedback
5. Developer addresses feedback — pushes updates to the same branch
6. Reviewers approve — the PR is merged into the main branch and the branch is deleted

Modern practice: PRs also trigger automated tests and linters (CI) — the code must pass these before a human even looks at it.

Section 6.6

Version Control with Git

The essential skill every software engineer must have

Why Version Control?

Without version control, collaboration is chaos

Without version control — what actually happens on a team project:
"I changed the login function" + "I also changed the login function" → whose version do we keep? What did each of us change? Who has the latest file — your email or mine? The demo is in 20 minutes and the code is broken. What did we change since yesterday?

What does version control solve?

Version control is a system that records every change made to every file over time. You can see who changed what, when, and why. You can go back to any previous version. Multiple people can work simultaneously without overwriting each other's work.

Version control gives you three superpowers:

History

Every change is recorded permanently. If you break something, you can go back. Nothing is ever truly lost.

Collaboration

Multiple developers work on different features simultaneously — version control merges their work together.

Safety

Experiment freely on a branch — if it goes wrong, delete the branch. The main code is untouched.

What is Git?

The most widely used version control system in the world

Git — a brief history

Git was created in 2005 by Linus Torvalds — the creator of Linux — because the team needed a version control system that could handle thousands of simultaneous contributors. Today it is used by virtually every software company in the world.

Key concepts you must understand:

Repository (repo)	The entire history of a project — every file and every change ever made — stored in a hidden .git folder
Commit	A saved snapshot of the project at one point in time. A commit includes what changed, who changed it, when, and a message explaining why
Branch	A parallel version of the project. You branch off to work on a feature, then merge back when done. The main branch is protected.
Remote	A copy of the repository stored on a server (GitHub, GitLab). Others push their changes there and pull yours from there.
Working directory	The files you see on your computer right now. Not yet saved to the repository until you commit them.

The Three Stages of a File in Git

Understanding where your file is at any moment

Every file in a Git project can be in one of three states. Understanding this is the foundation of working with Git.

Working Directory

Modified

You have edited the file but not yet told Git about it. Git sees the change but has not recorded it.

```
git status →
shows modified files
```

Staging Area (Index)

Staged

You have marked the file to be included in the next commit. This lets you choose exactly which changes to commit.

```
git add filename →
moves file here
```

Repository (.git)

Committed

The change is permanently saved in Git's history. It can always be retrieved.

```
git commit -m 'message' →
saves here
```

Core Git Commands — The Daily Workflow

These are the commands you will use every single day

git init	Create a new Git repository in the current folder. Run once at the start of a project.
git clone <url>	Download a complete copy of a remote repository to your machine. How you start working on an existing project.
git status	Show which files are modified, staged, or untracked. Run this constantly — it tells you where you are.
git add <file> or git add .	Move a file (or all files) to the staging area. The dot (.) stages everything changed.
git commit -m 'message'	Save the staged changes as a new commit. The message must explain WHAT changed and WHY.
git push	Send your local commits to the remote repository (GitHub). Others can now see your work.
git pull	Download and apply the latest commits from the remote repository. Do this before starting new work.
git log	See the full history of commits — who, when, and what message. Add --oneline for a compact view.

Branching — Working Safely in Parallel

A branch lets you experiment freely without affecting the main codebase

What is a branch?

A branch is a separate line of development. You create one to work on a feature or fix, make commits there freely, and then merge it back into the main branch when the work is complete and reviewed.

Branch commands:

Git branch commands

```
git branch                  # List all branches — current branch shown with *
git branch feature/fee-payment  # Create a new branch named feature/fee-payment
git checkout feature/fee-payment # Switch to that branch (your changes now go here)
git checkout -b feature/login   # Create AND switch in one command (shortcut)
git merge feature/fee-payment   # Merge the branch back into the current branch (usually main)
git branch -d feature/fee-payment  # Delete the branch after merging (clean up)
```

Branch naming conventions used in professional teams:

feature/ e.g. feature/user-login, feature/fee-payment — new functionality

fix/ e.g. fix/login-error, fix/null-pointer-crash — bug fixes

docs/ e.g. docs/api-reference — documentation updates

Git Commit Messages — Why They Matter

A well-written commit message is documentation that saves hours of detective work later

The rule: explain WHY, not just WHAT

A commit message should say why this change was made, not just describe what the code does — the code shows what it does. Future engineers (including you) will read these messages when debugging.

Bad commit messages vs Good commit messages:

✗ Bad	✓ Good
✗ fix	✓ Fix null pointer in EnrolmentManager when courseCode is empty
✗ changes	✓ Add late fee waiver for first offence (regulation 4.2.1)
✗ updated login	✓ Increase session timeout from 30 min to 60 min — user feedback issue #47
✗ asdfgh	✓ Refactor PaymentController — extract validation into separate method

Format for a good commit message:

Short summary (max 72 chars): WHAT changed · If needed, add more lines: WHY the change was needed

GitHub — Git in the Cloud

The world's largest software collaboration platform

Git vs GitHub — they are not the same thing
Git is the version control tool that runs on your computer. GitHub is a website that hosts Git repositories online, so teams can collaborate. GitLab and Bitbucket are popular alternatives to GitHub.

What GitHub gives you beyond Git:

Remote repository	Your code stored safely in the cloud — accessible from anywhere, backed up automatically
Pull Requests	The standard mechanism for code review — propose a change, discuss it, get it approved, merge it
Issues	Track bugs, feature requests, and tasks. Link issues to commits and pull requests
GitHub Actions	Automate workflows — run tests automatically when you push, deploy when you merge to main (CI/CD)
GitHub Copilot	AI pair programmer built into GitHub — suggests code as you type

`git remote add origin https://github.com/your-username/your-repo.git` → connect local repo to GitHub

A Complete Git Workflow — From Start to Merge

The sequence every professional developer follows for every piece of work

1	**Pull the latest code** `git pull origin main`	Always start by syncing with the remote — get everyone else's changes first
2	**Create a feature branch** `git checkout -b feature/register-for-course`	Never work directly on main — create a branch for your specific task
3	**Write code and test it** [edit files in your IDE]	Make your changes. Test them. Fix issues. Test again.
4	**Stage the changed files** `git add .` or `git add specific-file.py`	Select exactly which changes you want to commit
5	**Commit with a good message** `git commit -m 'Add course availability'`	Save a snapshot with a clear explanation
6	**Push the branch to GitHub** `git push origin feature/register-for-course`	Upload your branch to the remote so others can see it
7	**Open a Pull Request** [on GitHub — open PR]	Request a review. Your team reads the code, comments, approves
8	**Merge and delete branch** [merge on GitHub after approval]	Changes land in main. Delete the feature branch — it has served its purpose

Resolving Merge Conflicts

When two people change the same line — Git asks you to decide

What is a merge conflict?

A merge conflict happens when two developers change the same part of the same file independently. Git cannot automatically decide which version to keep — it marks both versions in the file and asks you to resolve it manually.

Three steps to resolve:

1. Open the conflicted file — Git marks conflicts with <<<<<< , ======= , and >>>>>>>
2. Edit the file to keep the correct version — delete the markers, combine if needed
3. git add the resolved file, then git commit — the merge is now complete

Modern IDEs (VS Code, IntelliJ) show conflicts visually with Accept Current / Accept Incoming / Accept Both buttons — much easier than editing raw text.

Git conflict markers in a file

```
<<<<<<< HEAD  (your version — currently on this branch)
def calculate_gpa(grades, credits):
    return sum(g*c for g,c in zip(grades, credits)) / sum(credits)
=======
(the divider between the two versions)
def calculate_gpa(grades, credits, scale=4.0):
    weighted = sum(g*c for g,c in zip(grades, credits))
    return round(weighted / sum(credits), 2)
>>>>>>>
feature/gpa-scale  (the incoming version from the other branch)
Resolve by: delete the markers, keep what is correct (or combine both), save, then git add → git commit
```

Section 6.7

Open Source and Licensing

Who owns the code — and what you can do with it

What is Open Source Software?

The foundation of modern software development — most systems you build will use it

Definition

Open source software is software whose source code is publicly available — anyone can read it, modify it, and distribute it, subject to the conditions of its licence. This is in contrast to proprietary (closed source) software, where only the owner can see and modify the code.

Why open source matters to you as a software engineer:

You already use it	Python, VS Code, Git, Linux, Node.js, React, PostgreSQL — all open source. You cannot build modern software without it.
Career infrastructure	GitHub is your professional portfolio. Open source contributions are reviewed by employers. This is your CV in code.
Legal obligation	Using open source code in your project creates legal obligations based on its licence. Ignoring this is a real risk.
Supply chain risk	Security vulnerabilities in open source libraries affect your software. You must track and update your dependencies.

Key Open Source Licences

Every open source project has a licence — read it before you use it

A licence tells you what you can and cannot do with open source code. Choosing the wrong licence — or ignoring one — can have serious legal consequences.

MIT Licence	Very permissive. Do almost anything — use commercially, modify, distribute. Must include the original copyright notice. Used by: React, jQuery, Rails
Apache 2.0	Permissive. Like MIT but includes a patent licence — protects users from patent claims by contributors. Used by: Kubernetes, TensorFlow, Android
GPL (General Public Licence)	Copyleft — 'viral'. If you distribute software that uses GPL code, your software must also be GPL and open source. Used by: Linux kernel, GCC
LGPL (Lesser GPL)	Weaker copyleft. You can use LGPL libraries in proprietary software without opening your code, but modifications to the library itself must be shared. Used by: GNU C Library
BSD Licence	Permissive. Similar to MIT. Very short. 3-clause version adds advertising restriction. Used by: FreeBSD, Nginx, Flask

Practical rule: for commercial projects, prefer MIT, Apache 2.0, or BSD.
Carefully evaluate any GPL dependency before including it.

Choosing a Licence for Your Own Project

When you publish code, choose a licence that reflects your intentions

When you publish your code without a licence, no one can legally use it. Always choose one.

I want maximum adoption — anyone can use it for any purpose	→ MIT or Apache 2.0
I want it open, but I also want patent protection for users	→ Apache 2.0
I want all derivative work to remain open source too	→ GPL — your code 'stays free'
I want companies to use my library but modifications stay open	→ LGPL
I want to keep it private and own all rights	→ Proprietary — do not publish without legal advice

choosealicense.com — GitHub's free resource to help you select the right licence for your project.

Section 6.8

AI in Implementation

The new paradigm — engineers who generate, direct, and validate

A Fundamental Shift in How Code is Written

The question is no longer only 'how do I write this?' — it is also 'how do I direct an AI to generate this?'

The old model

Engineer understands the problem → Engineer designs a solution → Engineer writes every line of code → Engineer tests it

The new model

Engineer understands the problem → Engineer crafts a prompt → AI generates code → Engineer reviews, tests, refines

The critical insight:

Generation is NOT the same as pressing a button and getting finished software. It is a new discipline — you must understand the problem deeply enough to direct the AI, evaluate what it produces, catch what it gets wrong, and take full professional responsibility for the result.

The skills that become MORE important with AI:

Deep problem understanding	You cannot direct an AI to solve a problem you do not understand. The engineer still defines the problem.
Code reading and critical evaluation	You receive AI-generated code. Can you read it? Does it actually do what the prompt said? Is it secure?
Testing and validation	AI-generated code can look correct and behave incorrectly. Rigorous testing is more important, not less.
Architecture and design judgement	AI generates components — you design how they fit together. Architecture remains a human activity.

AI Coding Tools — What Exists Today

The landscape of AI tools for software implementation in 2026

GitHub Copilot	Integrated into VS Code, JetBrains, and other IDEs. Suggests complete functions, methods, and tests as you type. Trained on billions of lines of public code. Subscription-based — now standard in most professional environments.
Cursor	An AI-first IDE built on VS Code. You describe what you want to build in natural language and Cursor writes, edits, and explains code. Can make changes across multiple files simultaneously.
ChatGPT and Claude	Conversational AI used for: explaining unfamiliar code, debugging, refactoring, generating tests, writing documentation, and answering 'how do I do X in Python?' questions.
Amazon CodeWhisperer	Amazon's AI coding tool — particularly effective for AWS cloud development. Free tier available. Integrates with VS Code and JetBrains.
Tabnine	AI code completion trained on your team's private codebase — learns your project's patterns, naming conventions, and coding style. Suitable for private enterprise code.

Prompt Engineering for Code Generation

The skill of communicating with AI effectively — getting useful output requires precision

What is prompt engineering?

Prompt engineering is the discipline of crafting input instructions to AI systems that produce useful, accurate, and well-structured output. For code generation, a good prompt is as important as a good design specification.

✗ Weak prompt

```
Write a function to check fees
```

Problems: no language specified, no context (what fees?), no inputs/outputs, no error handling mentioned, no style requirements

✓ Effective prompt

```
Write a Python function
check_outstanding_fees(student_id: str) -> dict that
queries the database and returns {'outstanding':
bool, 'amount': float, 'due_date': str}. Handle
StudentNotFoundError with a clear error message.
Follow PEP 8 style.
```

Specifies: language, function name, signature, return type, error handling, coding standard. AI now has everything it needs.

Rule: write a prompt as though you are writing a requirements specification for a junior developer. Precision produces precision.

What AI Does Well in Implementation

Use AI for these — confidently

AI assistance is most reliable for these specific implementation activities:

Boilerplate and scaffolding	Database connection setup, REST endpoint stubs, data model classes — repetitive structure that follows known patterns
Explaining unfamiliar code	Paste code you do not understand — AI explains it line by line, identifies patterns, and describes what each part does
Generating unit tests	Given a function, AI suggests test cases — including edge cases and error scenarios that humans often miss
Refactoring suggestions	Paste messy code and ask for a refactored version that is more readable, more efficient, or applies a specific pattern
Writing documentation	Docstrings, README files, API documentation — AI excels at generating clear prose from structured code
Debugging assistance	Describe a bug and paste the code — AI often identifies the likely cause, suggests a fix, and explains the reasoning

What AI Gets Wrong — Critical Awareness

AI-generated code requires rigorous professional review — always

AI-generated code can look completely correct and behave incorrectly.
You are responsible for everything you submit.

⚠ Hallucination — invented functions

AI confidently calls functions that do not exist. Common with libraries — it invents plausible-sounding API methods that are not real. Always verify library calls against actual documentation.

⚠ Security vulnerabilities

AI learns from all public code — including insecure code. SQL injection, cross-site scripting, insecure default configurations, and hardcoded credentials appear in AI-generated code. Always run a security linter.

⚠ Wrong logic — correct syntax

Code that compiles and runs but produces the wrong result for certain inputs. Unit tests written BY YOU (not generated by the same AI) catch this — never skip testing AI output.

⚠ Outdated patterns

AI training data has a cutoff. It may generate code for an outdated version of a library, use a deprecated API, or miss a security patch. Always check the library version.

⚠ Licence issues

AI trained on open source code may reproduce copyrighted code verbatim. In commercial projects, this creates legal risk. Use tools like GitHub Copilot's licence filter.

The Engineer's Responsibilities in an AI-Assisted World

What technology cannot do — and what you must still own professionally

The arrival of AI coding tools does not reduce your professional responsibility — it shifts WHERE you apply your engineering judgement.

You own the outcome	If AI-generated code causes a system failure, data loss, or security breach, the engineer who accepted it is accountable. Responsibility does not transfer to the AI.
You define the problem	AI generates solutions. You decide what problem to solve, what constraints apply, and whether the generated solution fits the context. This requires deep understanding.
You validate correctness	AI produces plausible code, not necessarily correct code. You must read, understand, test, and verify every piece of generated code before accepting it.
You make ethical decisions	AI has no professional ethics. Decisions about privacy, data protection, accessibility, fairness, and social impact remain entirely with the engineer.
You understand what you submit	Submitting code you do not understand is professional negligence — regardless of how it was generated. You must be able to explain every line you commit.

Section 6 — What We Have Covered

Check your understanding against the learning outcomes

LO1 Implementation transforms a design specification into executable code. The quality of implementation directly determines the cost of testing and long-term maintenance.

LO2 Programming paradigms: procedural, object-oriented, functional, declarative. Language choice is driven more by team expertise and context than by language features alone.

LO3 The development environment: IDEs (VS Code, IntelliJ), package managers (pip, npm), linters (ESLint, Pylint), and formatters (Prettier, Black) form the professional toolkit.

LO4 Good code is Reliable (fault avoidance + tolerance), Readable (naming, comments, self-documentation), and Reusable. SOLID principles and avoiding technical debt complete the picture.

LO5 Code review: walkthroughs, inspections, and pull requests. PRs are the standard modern workflow — code is reviewed inline on GitHub before merging to main.

LO6 Git: the three stages (working directory, staging area, repository), core commands (init, add, commit, push, pull), branching, merging, conflicts, and GitHub collaboration.

LO7 Open source licences: MIT and Apache (permissive), GPL (copyleft — derivative work must be open), LGPL (library use in proprietary code allowed). Always check the licence.

LO8 AI tools (Copilot, Cursor, ChatGPT) shift the paradigm from writing to directing and validating. Prompt engineering matters. Engineers retain full professional responsibility.

References and Further Reading

Main Textbooks

Software Engineering

Ian Sommerville, 10th edition, Addison-Wesley, 2015
Chapter 3 (Agile — XP practices), Chapter 8 (Testing and Implementation)

Software Engineering: A Practitioner's Approach

Roger S. Pressman, 9th edition, McGraw-Hill, 2020
Chapters 14–17: Implementation, testing, and software quality

Clean Code

Robert C. Martin (Uncle Bob), Prentice Hall, 2008
The definitive guide to readable, maintainable code — highly recommended for all developers

Online Resources

- git-scm.com — official Git documentation and free interactive tutorials
- learngitbranching.js.org — visual, interactive Git tutorial in the browser (free, highly recommended)
- choosealicense.com — GitHub's guide to choosing an open source licence
- docs.github.com/copilot — GitHub Copilot documentation and getting started guide
- refactoring.guru/refactoring — free guide to code smells and refactoring techniques

Coming Up Next

Section 7: Software Testing

Unit testing · Integration testing · System testing · Test-Driven Development · Testing tools

Tutorial this week:

Set up Git on your machine · Create a repo · Make 3 commits on a branch · Open a pull request · Try GitHub Copilot on a small function